

Homework 1: Functions, Control, Higher-Order Functions

题目来源: CS 61A Summer 2026 — <https://cs61a.org/hw/hw01/>

下载 hw01.zip。

提示

下载并解压后, 用 VS Code 打开 **hw01 文件夹** (点击文件 → 打开文件夹, 选择 hw01 文件夹), 而**不是**单独打开某个.py 文件。这样才能在终端中正确运行 Ok 评测命令。

在 hw01 文件夹中, 你只需要编辑 hw01.py 这一个文件来完成所有题目, 其他文件无需修改。

1 必做题 (Required Questions)

1.1 Functions & Control

Q1: A Plus Abs B

Python 的 `operator` 模块包含双参数函数, 如 `add` 和 `sub`, 对应 Python 内置的算术运算符。例如, `add(2, 3)` 求值为 5, 与表达式 `2 + 3` 一样。

在以下函数中填入空白, 实现将 `a` 与 `b` 的绝对值相加, 但**不能**调用 `abs` 函数。除了两个空白外, **不得**修改任何已有代码。

```
1 def a_plus_abs_b(a, b):
2     """Return a+abs(b), but without calling abs.
3
4     >>> a_plus_abs_b(2, 3)
5     5
6     >>> a_plus_abs_b(2, -3)
7     5
8     >>> a_plus_abs_b(-1, 4)
9     3
10    >>> a_plus_abs_b(-1, -4)
```

```

11     3
12     """
13     if b < 0:
14         f = -----
15     else:
16         f = -----
17     return f(a, b)

```

使用 Ok 测试代码:

```
python3 ok -q a_plus_abs_b
```

注意

第一次使用 Ok 评测时，终端可能会提示你输入邮箱账号，输入你自己的邮箱即可。后续再次使用 Ok 时就不会再出现这个提示了。

使用 Ok 运行本地语法检查（检查你是否未修改除两个空白之外的任何代码）:

```
python3 ok -q a_plus_abs_b_syntax_check
```

Q2: Hailstone

Douglas Hofstadter 的普利策奖获奖著作 *Gödel, Escher, Bach* 中提出了以下数学谜题:

1. 选择一个正整数 n 作为起始值。
2. 如果 n 是偶数，将其除以 2。
3. 如果 n 是奇数，将其乘以 3 再加 1。
4. 持续这个过程，直到 n 变为 1。

数字 n 会上下波动，但最终都会变为 1（至少对所有曾尝试过的数字都是如此——尽管没有人证明过该序列一定会终止）。类似地，冰雹（hailstone）在大气中上下运动，最终落回地面。

这种 n 的值序列通常被称为 Hailstone 序列。请编写一个函数，接收一个形参名为 n 的参数，打印从 n 开始的 hailstone 序列，并返回序列的步数:

```

1 def hailstone(n):
2     """Print the hailstone sequence starting at n and return its
3     length.

```

```

4
5     >>> a = hailstone(10)
6     10
7     5
8     16
9     8
10    4
11    2
12    1
13    >>> a
14    7
15    >>> b = hailstone(1)
16    1
17    >>> b
18    1
19    ""
20    "*** YOUR CODE HERE ***"

```

Hailstone 序列可以非常长！试试 27。你能找到的最长序列是多少？

注意

如果初始时 $n = 1$ ，则序列的长度为 1 步。

提示：如果你看到 4.0 但想要 4，尝试使用地板除法 `//` 而不是普通除法 `/`。

使用 Ok 测试代码：

```
python3 ok -q hailstone
```

对 hailstone 序列感兴趣？看看这篇文章：

- 2019 年，在理解 hailstone 猜想对大多数数字的适用性方面取得了重大进展！

1.2 Higher-Order Functions

Q3: Product

编写一个名为 `product` 的函数，返回一个序列前 n 项的乘积。具体来说，`product` 接收一个整数 n 和 `term`（一个单参数函数，用于确定一个序列——即 `term(i)` 给出该序列的第 i 项）。`product(n, term)` 应返回 $\text{term}(1) \times \dots \times \text{term}(n)$ 。

```

1 def product(n, term):
2     """Return the product of the first n terms in a sequence.

```

```

3
4     n: a positive integer
5     term: a function that takes an index as input and produces a term
6
7     >>> product(3, identity) # 1 * 2 * 3
8     6
9     >>> product(5, identity) # 1 * 2 * 3 * 4 * 5
10    120
11    >>> product(3, square)    # 1^2 * 2^2 * 3^2
12    36
13    >>> product(5, square)    # 1^2 * 2^2 * 3^2 * 4^2 * 5^2
14    14400
15    >>> product(3, increment) # (1+1) * (2+1) * (3+1)
16    24
17    >>> product(3, triple)    # 1*3 * 2*3 * 3*3
18    162
19    """
20    """*** YOUR CODE HERE ***"""

```

使用 Ok 测试代码:

```
python3 ok -q product
```

Q4: Make Repeater

实现函数 `make_repeater`，它接收一个单参数函数 f 和一个正整数 n 。它返回一个单参数函数，使得 `make_repeater(f, n)(x)` 返回将 f 应用于 x 共 n 次的值，即 $f(f(\dots f(x)\dots))$ 。例如，`make_repeater(square, 3)(5)` 对 5 进行三次平方运算并返回 390625，就像 `square(square(square(5)))` 一样。

```

1 def make_repeater(f, n):
2     """Returns the function that computes the nth application of f.
3
4     >>> add_three = make_repeater(increment, 3)
5     >>> add_three(5)
6     8
7     >>> make_repeater(triple, 5)(1) # 3 * (3 * (3 * (3 * (3 * 1))))
8     243
9     >>> make_repeater(square, 2)(5) # square(square(5))
10    625

```

```

11 >>> make_repeater(square, 3)(5) # square(square(square(5)))
12 390625
13 ""
14 "*** YOUR CODE HERE ***"

```

使用 Ok 测试代码：

```
python3 ok -q make_repeater
```

本地检查分数 (Check Your Score Locally)

你可以通过运行以下命令在本地检查每道题目的得分：

```
python3 ok --score
```

2 选做题 (Optional Questions)

Q5: Largest Factor

编写一个函数，接收一个大于 1 的整数 n ，返回小于 n 且能整除 n 的最大整数。

```

1 def largest_factor(n):
2     """Return the largest factor of n that is smaller than n.
3
4     >>> largest_factor(15) # factors are 1, 3, 5
5     5
6     >>> largest_factor(80) # factors are 1, 2, 4, 5, 8, 10, 16, 20, 40
7     40
8     >>> largest_factor(13) # factors are 1, 13
9     1
10    """
11    "*** YOUR CODE HERE ***"

```

提示

要检查 b 是否能整除 a ，使用表达式 $a \% b == 0$ ，可以解读为「 a 除以 b 的余数为 0」。

使用 Ok 测试代码：

```
python3 ok -q largest_factor
```

Q6: Accumulate

让我们来看看 `product` 如何是一个更通用的函数 `accumulate` 的特例，我们需要实现它：

```

1 def accumulate(fuse, start, n, term):
2     """Return the result of fusing together the first n terms in a
3     sequence
4     and start. The terms to be fused are term(1), term(2), ..., term(n).
5     The function fuse is a two-argument commutative & associative
6     function.
7
8     >>> accumulate(add, 0, 5, identity) # 0 + 1 + 2 + 3 + 4 + 5
9     15
10    >>> accumulate(add, 11, 5, identity) # 11 + 1 + 2 + 3 + 4 + 5
11    26
12    >>> accumulate(add, 11, 0, identity) # 11 (fuse is never used)
13    11
14    >>> accumulate(add, 11, 3, square) # 11 + 1^2 + 2^2 + 3^2
15    25
16    >>> accumulate(mul, 2, 3, square) # 2 * 1^2 * 2^2 * 3^2
17    72
18    >>> # 2 + (1^2 + 1) + (2^2 + 1) + (3^2 + 1)
19    >>> accumulate(lambda x, y: x + y + 1, 2, 3, square)
20    19
    """
    """*** YOUR CODE HERE ***"""

```

`accumulate` 具有以下参数：

- **fuse**: 一个双参数函数，指定如何将当前项与之前累积的项融合在一起
- **start**: 开始累积的初始值
- **n**: 一个非负整数，表示要融合的项数
- **term**: 一个单参数函数；`term(i)` 是序列的第 i 项

实现 `accumulate`，使用 `fuse` 函数将由 `term` 定义的序列的前 n 项与 `start` 值融合在一起。

例如, `accumulate(add, 11, 3, square)` 的结果是:

$$\begin{aligned} & \text{add}(11, \text{add}(\text{square}(1), \text{add}(\text{square}(2), \text{square}(3)))) \\ &= 11 + \text{square}(1) + \text{square}(2) + \text{square}(3) \\ &= 11 + 1 + 4 + 9 = 25 \end{aligned}$$

假设

假设 `fuse` 是交换的, 即 $\text{fuse}(a, b) = \text{fuse}(b, a)$, 并且是结合的, 即 $\text{fuse}(\text{fuse}(a, b), c) = \text{fuse}(a, \text{fuse}(b, c))$ 。

使用 Ok 测试代码:

```
python3 ok -q accumulate
```

然后, 将 `summation_using_accumulate` 和 `product_using_accumulate` 实现为对 `accumulate` 的一行调用。

重要

`summation_using_accumulate` 和 `product_using_accumulate` 都应仅用一行代码实现, 以 `return` 开头。

```
1 def summation_using_accumulate(n, term):
2     """Returns the sum: term(1) + ... + term(n), using accumulate.
3
4     >>> summation_using_accumulate(5, square) # square(1)+square(2)+...+
5         square(5)
6         55
7     >>> summation_using_accumulate(5, triple) # triple(1)+triple(2)+...+
8         triple(5)
9         45
10    >>> # This test checks that the body of the function is just a return
11        statement.
12    >>> import inspect, ast
13    >>> [type(x).__name__ for x in ast.parse(inspect.getsource(
14        summation_using_accumulate)).body[0].body]
15    ['Expr', 'Return']
16    """
17    return ____
18
19 def product_using_accumulate(n, term):
```

```
16     """Returns the product: term(1) * ... * term(n), using accumulate.
17
18     >>> product_using_accumulate(4, square) # square(1)*square(2)*square
19         (3)*square(4)
20         576
21     >>> product_using_accumulate(6, triple) # triple(1)*triple(2)*...*
22         triple(6)
23         524880
24     >>> # This test checks that the body of the function is just a return
25         statement.
26     >>> import inspect, ast
27     >>> [type(x).__name__ for x in ast.parse(inspect.getsource(
    product_using_accumulate)).body[0].body]
    ['Expr', 'Return']
    """
    return ----
```

使用 Ok 测试代码:

```
python3 ok -q summation_using_accumulate
python3 ok -q product_using_accumulate
```